

Unit_II_Chapter_III

Modules: Modules and Files, Namespaces, Importing Modules, Importing Module Attributes, Module Built-in Functions, Packages, Other Features of Modules.

Modules and Files

Python programs can quickly grow large and complex. To manage this, Python provides two essential organizational tools: **modules** (to structure code) and **files** (to store and persist data).

Modules in Python

Why use Modules?

- Reusability → Write once, use anywhere.
- Organization → Break large programs into smaller parts.
- Namespace separation → Avoids naming conflicts. Example: (`math.sqrt` vs. `numpy.sqrt`).
- Community → Access to thousands of pre-built modules.

Types of Modules

Built-in Modules (Standard Library)

- Already included with Python.

```
1  import math
2  print(math.sqrt(16))    # 4.0
3  print(math.pi)         # 3.141592653589793
```

- Other examples: os, sys, datetime, random, json, re, http, etc.

User-defined Modules

- You create your own module by saving functions/classes in a .py file.
- Example: **mymodule.py**

```
def greet(name):  
    return f"Hello, {name}!"
```

main.py

```
import mymodule  
print(mymodule.greet("Neel"))
```

Third-party Modules

Installed via pip.

- Example:

pip install requests pandas flask

```
1  import requests
2  response = requests.get("https://api.github.com")
3  print(response.json())
```

Importing Modules

```
1  import math                # full module
2  print(math.factorial(5))
3
4  from math import sqrt, pi  # specific functions
5  print(sqrt(25), pi)
6
7  import math as m           # alias
8  print(m.log(100, 10))
9
10 from math import *          # all functions (not recommended)
11 print(sin(pi/2))
```

Files in Python

Files allow us to **store and retrieve data permanently** on disk (unlike variables which disappear when a program ends).

File Operations

Python uses the built-in `open()` function.

```
1  f = open("test.txt", "w")    # open for writing
2  f.write("Hello, World!")
3  f.close()
```

Modes:

- `"r"` → read (default)
- `"w"` → write (overwrites existing file)
- `"a"` → append
- `"b"` → binary mode
- `"r+"` → read & write

Reading Files

```
f = open("test.txt", "r")
print(f.read())          # entire file
f.close()
```

Other useful methods:

```
f.readline()             # read one line
f.readlines()            # read all lines as list
for line in f:           # iterate line by line
    print(line.strip())
```

Writing Files

```
with open("data.txt", "w") as f:    # auto-close
    f.write("Line 1\n")
    f.write("Line 2\n")
```

```
with open("data.txt", "a") as f:    # append
    f.write("Line 3\n")
```

Working with JSON (structured data)

```
1  import json
2
3  data = {"name": "Neel", "age": 21}
4
5  # Write JSON
6  with open("data.json", "w") as f:
7      |    json.dump(data, f)
8
9  # Read JSON
10 with open("data.json", "r") as f:
11     |    new_data = json.load(f)
12
13 print(new_data)
```

CSV Files

```
1  import csv
2
3  # Writing CSV
4  with open("students.csv", "w", newline="") as f:
5      writer = csv.writer(f)
6      writer.writerow(["Name", "Age"])
7      writer.writerow(["Neel", 21])
8
9  # Reading CSV
10 with open("students.csv", "r") as f:
11     reader = csv.reader(f)
12     for row in reader:
13         print(row)
14
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

```
PS C:\Users\Acer> python -u "C:\Users\Acer\AppData\Local\Temp\
['Name', 'Age']
['Neel', '21']
PS C:\Users\Acer>
```

Modules + Files Together

Often, modules interact with files. For example:

- **Database module (sqlite3)** → stores/retrieves data in a file-based database.
- **Pandas** → reads/writes Excel, CSV, JSON files.
- **OS module** → manages file system paths and directories.

```
1  import os
2
3  if os.path.exists("test.txt"):
4      os.remove("test.txt")
5  else:
6      print("File not found")
7
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

```
PS C:\Users\Acer> python -u "C:\Users\Acer\AppData
File not found
PS C:\Users\Acer> █
```

Various python modules

1. Built-in Standard Library Modules (come with Python)

These are pre-installed and don't require external installation.

System & OS Interaction

- **os** → File and directory operations
- **sys** → System-specific functions, command-line arguments
- **platform** → Information about the underlying platform
- **shutil** → High-level file operations (copy, move, remove)
- **subprocess** → Run system commands

Math & Numbers

- **math** → Basic math functions (sin, cos, sqrt, etc.)
- **cmath** → Complex number mathematics
- **decimal** → High-precision decimal numbers
- **fractions** → Fractions handling
- **random** → Random number generation
- **statistics** → Mean, median, mode, variance, etc.

Data & Collections

- **collections** → Advanced data structures (deque, Counter, defaultdict)
- **array** → Efficient arrays of basic values
- **heapq** → Heap (priority queue) algorithms
- **queue** → FIFO/LIFO queues

Date & Time

- **datetime** → Date and time handling
- **time** → Time access and conversions
- **calendar** → Calendar-related functions

File & Data Formats

- **json** → Working with JSON data
- **csv** → Reading and writing CSV files
- **pickle** → Object serialization
- **xml.etree.ElementTree** → XML parsing
- **configparser** → INI-style configuration files

Networking & Internet

- **socket** → Low-level networking
- **http.client** → HTTP protocol client
- **urllib** → Fetching URLs
- **ftplib** → FTP protocol
- **smtplib** → Sending emails with SMTP

Compression & Archiving

- **zipfile, tarfile, gzip, bz2** → Working with compressed files

Concurrency & Parallelism

- **threading** → Multithreading
- **multiprocessing** → Parallel processing
- **asyncio** → Asynchronous programming

Debugging & Development

- **logging** → Logging messages
- **traceback** → Printing stack traces
- **pdb** → Debugger

Others

- **re** → Regular expressions
- **hashlib** → Hash functions (SHA, MD5, etc.)
- **uuid** → Unique IDs
- **functools** → Higher-order functions, caching
- **itertools** → Iterators and combinatorial tools

Popular External Python Modules

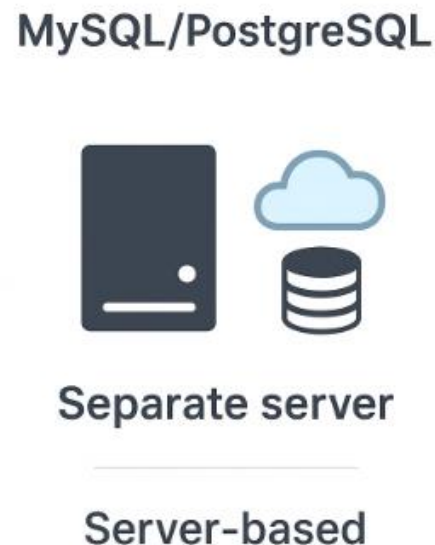
(need installation via `pip install module_name`)

Data Science & Machine Learning

- **numpy** – numerical computing
- **pandas** – data manipulation and analysis
- **matplotlib** – plotting and visualization
- **seaborn** – statistical data visualization
- **scikit-learn** – machine learning
- **tensorflow / torch** – deep learning
- **scipy** – scientific computing

Databases

- **sqlalchemy** – Object Relational Mapping(ORM) for relational databases
- **pymysql** – MySQL database connector
- **psycopg2** – PostgreSQL connector
- **sqlite3** – SQLite (comes built-in)
- **tinydb** – NoSQL lightweight database



Web Development

- **flask** – lightweight web framework
- **django** – full-stack web framework
- **fastapi** – modern, fast API framework
- **requests** – HTTP requests library
- **beautifulsoup4** – web scraping
- **scrapy** – web crawling

GUI Programming

- **tkinter** – built-in GUI toolkit
- **PyQt5 / PySide2** – advanced GUI frameworks
- **kivy** – cross-platform GUI apps

Automation & Tools

- **selenium** – browser automation
- **pyautogui** – GUI automation
- **openpyxl** – Excel file handling
- **reportlab** – PDF generation

Others

- **PIL / pillow** – image processing
- **moviepy** – video editing
- **pygame** – game development
- **cryptography** – encryption tools

Namespace

- A **namespace** in Python is like a dictionary that maps names **names (identifiers /variables /functions /classes)** to their corresponding **objects (values)**.
- It helps avoid **naming conflicts** by ensuring that names are unique within their scope.
- Example: You can have a variable `x` inside a function and another `x` outside the function (global), and they won't clash because they belong to different namespaces.

Types of Namespaces in Python

Python has **four main types of namespaces**:

1. Built-in Namespace:

- Contains names that are **predefined** in Python.

Examples: `len()`, `id()`, `print()`, `int`, `str`.

- Always available in any program.

```
print(len([1, 2, 3])) # 'len' comes from built-in namespace
```

2. Global Namespace

- Created when a Python script or module is executed.
- Includes names like variables, functions, and classes defined at the top-level of the script.
- Accessible throughout the file (except inside a local function unless declared global).

```
1  x = 10  # global namespace
2
3  def show():
4      |    print(x)  # accessing global variable
5
6  show()
7  
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

```
PS C:\Users\Acer> python -u "C:\Users\Acer\AppData\Local\Temp\10
```

```
PS C:\Users\Acer>
```

```
1  x = 10
2
3  def change():
4      x = x + 1 # ERROR: Python thinks you are creating a local variable x
5      print("Inside function, x =", x)
6
7  change()
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

PS C:\Users\Acer> python -u "C:\Users\Acer\AppData\Local\Temp\tempCodeRunnerFile.python"

Traceback (most recent call last):

File "C:\Users\Acer\AppData\Local\Temp\tempCodeRunnerFile.python", line 7, in <module>
 change()

File "C:\Users\Acer\AppData\Local\Temp\tempCodeRunnerFile.python", line 4, in change
 x = x + 1 # ERROR: Python thinks you are creating a local variable x
 ^

UnboundLocalError: cannot access local variable 'x' where it is not associated with a value

PS C:\Users\Acer>

```
1  x = 10
2
3  def change():
4      global x # Tell Python: use the global x, not a new local x
5      x = x + 1
6      print("Inside function, x =", x)
7
8  change()
9  print("Outside function, x =", x)
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

```
PS C:\Users\Acer> python -u "C:\Users\Acer\AppData\Local\Temp\tempCodeRunnerFile.python"
Inside function, x = 11
Outside function, x = 11
PS C:\Users\Acer>
```

3. Enclosed Namespace (Nonlocal / Nested)

- Exists when functions are **nested inside another function**.
- The inner function can access variables of the outer function.
- This is also called the **nonlocal** namespace.

```
1  def outer():
2      y = 20 # enclosed namespace
3
4      def inner():
5          print(y) # can access outer variable
6      inner()
7
8  outer()
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

```
PS C:\Users\Acer> python -u "C:\Users\Acer\AppData\Local\Temp\
20
```

```
PS C:\Users\Acer>
```

4. Local Namespace

- Created inside a function whenever it is called.
- Contains names of function parameters and variables declared inside that function.
- Destroyed once the function exits.

```
1  def greet():
2      message = "Hello, Python!"    # local variable
3      print(message)
4
5  greet()
6  #print(message)    # ❌ Error: message not defined outside
```

```
1  def square(num):
2      result = num * num    # both num and result are local
3      return result
4
5  print(square(5))          #  Works fine
6  # print(num)              #  Error: num not defined globally
7
```

Modules and Namespaces

- A **module** in Python is simply a file containing Python code (variables, functions, classes, etc.).
- Each module has its **own namespace** (separate from others).
- This separation avoids **naming conflicts** between different modules.

Suppose we have two files:

math_ops.py

```
1  # This is math_ops module
2  x = 100
3
4  def add(a, b):
5      |     return a + b
```

main.py

```
1  import math_ops
2
3  print(math_ops.x)           # Access variable from math_ops namespace
4  print(math_ops.add(10, 20)) # Access function from math_ops namespace
```

Why Module Namespaces Are Useful

1. Avoids name clashes

Example: If two modules both define a function `add`, no problem, because they live in different namespaces.

```
1  import math_ops
2  import string_ops
3
4  math_ops.add(5, 10)      # Calls add() from math_ops
5  string_ops.add("Hi", " There") # Calls add() from string_ops
```

2. Provides structure

Namespaces make large projects manageable by organizing code into modules and packages.

Importing Module Attributes

- A **module** = a Python file (.py) containing functions, classes, and variables.
- The **attributes** of a module are the things defined inside it (functions, variables, classes, constants, etc.).
- You can import **the whole module**, or just **specific attributes**.

1. Import the Whole Module

import math

2. Import Specific Attributes from a Module

from math import sqrt, pi

3. Import All Attributes

*from math import **

4. Import with Alias (Short Name)

import math as m

5. Import Specific Attribute with Alias

from math import factorial as fact

Module Built-in Functions

When you **import** a module, Python provides some **built-in functions** to work with it. These functions let you **inspect, reload, and check attributes** of a module.

Common Built-in Functions for Modules:

1. **dir()** → List attributes in a module

```
1  import math
2  print(dir(math))    # Lists all functions, constants, classes in math
```

```
PS C:\Users\Acer> python -u "C:\Users\Acer\AppData\Local\Temp\tempCodeRunnerFile.python"
```

```
['__doc__', '__loader__', '__name__', '__package__', '__spec__', 'acos', 'acosh', 'asin', 'asinh', 'atan', 'atan2', 'atanh', 'cbrt', 'ceil', 'comb', 'copysign', 'cos', 'cosh', 'degrees', 'dist', 'e', 'erf', 'erfc', 'exp', 'exp2', 'expm1', 'fabs', 'factorial', 'floor', 'fmod', 'frexp', 'fsum', 'gamma', 'gcd', 'hypot', 'inf', 'isclose', 'isfinite', 'isinf', 'isnan', 'isqrt', 'lcm', 'ldexp', 'lgamma', 'log', 'log10', 'log1p', 'log2', 'modf', 'nan', 'nextafter', 'perm', 'pi', 'pow', 'prod', 'radians', 'remainder', 'sin', 'sinh', 'sqrt', 'sumprod', 'tan', 'tanh', 'tau', 'trunc', 'ulp']
```

```
PS C:\Users\Acer>
```

2. `help()` → Documentation of module or attribute

```
1 import math
2 help(math.sqrt)    # Help on sqrt function
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

```
PS C:\Users\Acer> python -u "C:\Users\Acer\AppData\Local\Temp\tempCodeRunnerFile.python"
Help on built-in function sqrt in module math:
```

```
sqrt(x, /)
```

```
    Return the square root of x.
```

```
PS C:\Users\Acer> █
```

3. **type()** → Check type of the module

```
1  import math
2  print(type(math))
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

```
PS C:\Users\Acer> python -u "C:\Users\Acer\AppData\Local\Te
<class 'module'>
PS C:\Users\Acer>
```

4. id() → Memory location of a module object

```
1  import math
2  print(id(math))    # Unique identity of module object
3
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

```
PS C:\Users\Acer> python -u "C:\Users\Acer\AppData\Local\Temp\tem
1302641271936
```

```
PS C:\Users\Acer> █
```

5. `globals()` & `locals()` → Check namespaces

```
1  import math
2
3  print("Globals: ", 'math' in globals()) # True
4  def test():
5      import random
6      print("Locals: ", 'random' in locals()) # True inside function
7
8  test()
```

PROBLEMS

OUTPUT

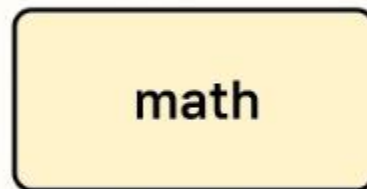
DEBUG CONSOLE

TERMINAL

PORTS

```
PS C:\Users\Acer> python -u "C:\Users\Acer\AppData\Local\Temp\tempCodeRunnerFi]
Globals: True
Locals: True
PS C:\Users\Acer> █
```

Global Namespace



Globals()



def test() :

random

Locals()

6. `__name__` → Module's name

- In Python, every module has a special built-in attribute called `__name__`.
- When you import `math`, the module is loaded, and its `__name__` attribute holds the string `"math"`.
- So `print(math.__name__)` outputs: `math`

This is useful to identify which module is being referenced, especially when debugging or working with multiple imports.

module_a.py

```
print("module_a is running. __name__ =", __name__)
```

module_b.py

```
print("module_b is running. __name__ =", __name__)
```

main.py

```
1  import module_a
2  import module_b
3  print("main.py is running. __name__ =", __name__)
```

```
module_a is running. __name__ = module_a
module_b is running. __name__ = module_b
main.py is running. __name__ = __main__
```

7. `__doc__` → Module's documentation string

```
1  import math
2  print(math.__doc__)
3
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

```
PS C:\Users\Acer> python -u "C:\Users\Acer\AppData\Loc
This module provides access to the mathematical functi
defined by the C standard.
PS C:\Users\Acer> █
```

8. `__file__` → Path of module (for non-built-in modules)

```
1  import os
2  print(os.__file__)    # Shows where the os module is stored
3
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

```
PS C:\Users\Acer> python -u "C:\Users\Acer\AppData\Local\Temp\tempCode
RunnerFile.python"
```

```
C:\Program Files\WindowsApps\PythonSoftwareFoundation.Python.3.12_3.12
.2800.0_x64__qbz5n2kfra8p0\Lib\os.py
```

```
PS C:\Users\Acer>
```

9. **importlib.reload()** → Reloads a module

- In Python, once a module is **imported**, it's stored (cached) in **sys.modules**.
- If you try to import it again, Python **doesn't reload it**, it just reuses the cached copy.
- This means any changes you make to the module file **won't take effect** until you restart Python.
- That's where **importlib.reload()** comes in: it forces Python to reload the module from the source file.

What is a Package?

- A **package** in Python is a way of organizing related modules into a directory hierarchy.
- A **module** is a single Python file (.py).
- A **package** is a **directory** that contains an **`__init__.py`** file (this tells Python that the directory should be treated as a package).
- Packages help in structuring large programs into **reusable, manageable parts**.

Structure of a Package

```
mypackage/  
|  
├─ __init__.py      # Initializes the package (can be empty or contain setup code)  
├─ module1.py        # Module 1  
├─ module2.py        # Module 2  
|  
└─ subpackage/  
    ├─ __init__.py  
    ├─ submodule1.py  
    └─ submodule2.py
```

Creating a Package

- Create a folder with the package name.
- Inside it, add an **`__init__.py`** file.
- Add your modules (.py files).

Importing from Packages

```
1  # Importing a whole module
2  import mypackage.module1
3
4  mypackage.module1.functionA()
5
6  # Import specific function/class
7  from mypackage.module1 import functionA
8
9  functionA()
10
11 # Import all items from a module
12 from mypackage.module1 import *
```

`__init__.py` Role

- Runs when the package is imported.
- Can define what gets imported with `*` using `__all__`.
- Example:

```
1  # mypackage/__init__.py
2  __all__ = ["module1", "module2"]
3  print("mypackage is imported")
```

```
1 import mypackage.module1 as m1
2 from mypackage.module2 import farewell
3 print(m1.greet("Alice"))
4 print(farewell("Bob"))
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

```
PS C:\Users\Acer> python -u "c:\Users\Acer\Desktop\Python
_PPT_B.Tech_Syllabus\chapter_3_unit_2\main.py"
```

```
Hello, Alice! Welcome from Module 1.
```

```
Goodbye, Bob! See you again from Module 2.
```

```
PS C:\Users\Acer>
```

```
1 def greet(name):  
2     return f"Hello, {name}! Welcome from Module 1."  
3
```

```
1 def farewell(name):  
2     return f"Goodbye, {name}! See you again from Module 2."
```

Extended Project Structure

```
demo_project/  
|— main.py  
└— mypackage/  
    |— __init__.py  
    |— module1.py  
    |— module2.py  
    └— subpackage/  
        |— __init__.py  
        |— submodule.py
```